

República Bolivariana de Venezuela
Ministerio del Poder Popular para la Educación Universitaria
Universidad Nacional Experimental de las Telecomunicaciones e Informática
Programación III MOD 1 Sección 6B

Evaluación Práctica. Desarrollo de Aplicación Móvil con Ionic

Profesor:

Msc. Carlos Márquez

Estudiante:

Alcides Mata C.I: V-31.158.186

07/05/2026

ÍNDICE DE ANEXOS

Foto I. Estado inicial de la aplicación previo a la refactorización. Autor: Alcides Mata	6
Foto II. Estructura de archivos tras la generación de las tres páginas independientes. Autor: Alcides Mata.....	7
Foto III. app-routing.module.ts junto con explicación de su funcionamiento. Autor: Alcides Mata	8
Foto IV. Verificación de la ruta /inicio funcional en el navegador. Autor: Alcides Mata.....	9
Foto V. Verificación de la ruta /informacion-personal funcional en el navegador. Autor: Alcides Mata.....	10
Foto VI. Verificación de la ruta /contacto funcional en el navegador. Autor: Alcides Mata	10
Foto VII. Página de Inicio con diseño final aplicado. Autor: Alcides Mata.....	12
Foto VIII. Página de Información Personal con diseño final aplicado. Autor: Alcides Mata	13
Foto IX. Página de Contacto con diseño final aplicado. Autor: Alcides Mata.....	14
Foto X. Vista responsiva móvil de la Página de Inicio. Autor: Alcides Mata	16
Foto XI. Vista responsiva móvil de la Página de Información Personal. Autor: Alcides Mata	17
Foto XII. Vista responsiva móvil de la Página de Contacto. Autor: Alcides Mata	18

INTRODUCCIÓN	4
Desarrollo de Aplicación Móvil con Ionic: Implementación de Menú Lateral con Tres Vistas.....	5
1. Configuración Inicial y Generación de las Páginas	5
2. Generación de las Páginas Independientes	6
Fase 2: Estructura de Rutas y Navegación	8
1. Configuración del Módulo de Enrutamiento.....	8
2. Verificación de Rutas en el Navegador.....	9
Fase 3: Implementación de las Vistas.....	11
1. Página de Inicio.....	11
2. Página de Información Personal.....	12
3. Página de Contacto.....	14
Fase 4: Validación Responsiva en Dispositivos Móviles	16
CONCLUSIÓN.....	20
ANEXOS.....	21
REFERENCIAS.....	22

INTRODUCCIÓN

El desarrollo de aplicaciones móviles modernas requiere el dominio de frameworks que permitan construir interfaces multiplataforma a partir de tecnologías web estándar. En el presente informe práctico se aborda la creación de una aplicación móvil mediante el uso de Ionic Framework integrado con Angular, dos tecnologías que en conjunto permiten desarrollar experiencias de usuario consistentes tanto en navegadores web como en dispositivos móviles nativos.

Ionic se define como un kit de desarrollo de código abierto orientado a la construcción de aplicaciones móviles híbridas, ofreciendo una colección de componentes de interfaz prefabricados que siguen las pautas de diseño de iOS y Android. Su capacidad de integración con Angular permite estructurar aplicaciones robustas mediante el patrón de módulos, componentes y servicios, mientras que Capacitor actúa como puente para empaquetar el código web hacia plataformas nativas.

Resulta pertinente mencionar que esta actividad se desarrolló en dos etapas. Una primera versión fue construida hace aproximadamente dos semanas utilizando la plantilla starter de Ionic con menú lateral, donde las tres secciones se renderizaban dentro de una sola página genérica mediante condicionales. Para la entrega final que aquí se documenta, se realizó una refactorización completa del proyecto; se generaron tres páginas independientes con sus respectivos módulos, se reescribió el sistema de rutas, y se aplicó un nuevo diseño visual a cada vista. Para esta refactorización se hizo uso del entorno de desarrollo AntigraVity de Google, asistido por el modelo Gemini en conjunto con el skill de diseño Impeccable, lo cual permitió acelerar la implementación manteniendo el control sobre las decisiones arquitectónicas y la documentación del código.

El objetivo de este documento es detallar, paso a paso, el proceso de construcción de la aplicación, desde la configuración inicial hasta la validación del comportamiento responsivo en móviles, mostrando las decisiones técnicas tomadas y las evidencias visuales correspondientes a cada etapa.

Desarrollo de Aplicación Móvil con Ionic: Implementación de Menú Lateral con Tres Vistas

Se solicitó como evaluación práctica de la unidad curricular el desarrollo de una aplicación móvil que cumpliera el siguiente requerimiento:

“Crear una aplicación con IONIC y Angular que contenga 3 menús: Inicio, Información Personal y Contacto.”

Es decir, se desarrolló una aplicación móvil híbrida con un menú lateral funcional que permite al usuario navegar entre tres secciones independientes, cada una con propósito y contenido visual diferenciado. A continuación, se describe el proceso completo dividido en cuatro fases consecutivas.

1. Configuración Inicial y Generación de las Páginas

Antes de comenzar la refactorización, se partió de un proyecto Ionic ya inicializado que contenía la plantilla starter de menú lateral. En esta plantilla original, las tres secciones (Inicio, Información Personal y Contacto) se mostraban dentro de una sola página llamada “folder”, utilizando bloques condicionales `@if` y `@else if` sobre el parámetro de URL para alternar el contenido visible. Si bien esta solución era visualmente funcional, no demostraba dominio en la generación de páginas independientes en Ionic, por lo que se decidió reestructurar el proyecto desde su base.

En la siguiente captura se puede apreciar el estado inicial del proyecto previo a cualquier modificación, donde se observa el menú lateral funcional pero apuntando a una única página con parámetros de URL:

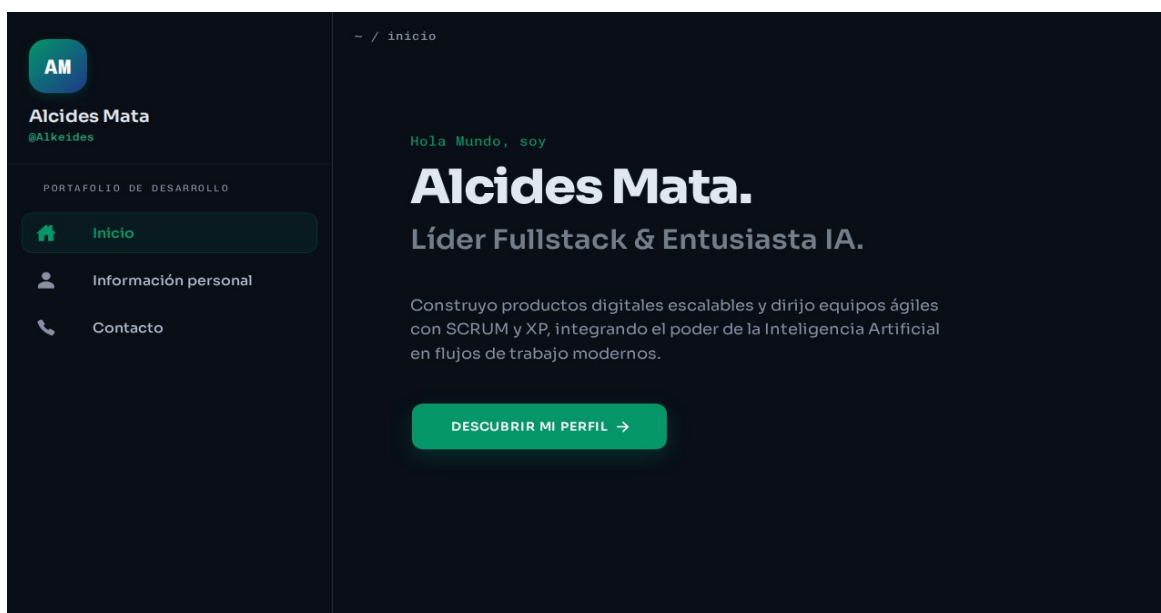


Foto I. Estado inicial de la aplicación previo a la refactorización. Autor: Alcides Mata

2. Generación de las Páginas Independientes

Utilizando la herramienta de línea de comandos de Ionic CLI, se generaron tres páginas independientes mediante el comando `ionic generate page`, el cual crea automáticamente la estructura completa de cada página: el módulo principal, el módulo de enrutamiento, el archivo de lógica TypeScript, la plantilla HTML, los estilos SCSS y el archivo de pruebas unitarias. Los comandos ejecutados fueron:

```
ionic generate page inicio
ionic generate page informacion-personal
ionic generate page contacto
```

El resultado de estas tres ejecuciones fue la creación de tres carpetas dentro del directorio `src/app/`, cada una conteniendo seis archivos generados automáticamente por el CLI. Esta estructura permite que cada página sea un módulo autónomo, lo que facilita el mantenimiento, la carga perezosa y la escalabilidad futura del proyecto.

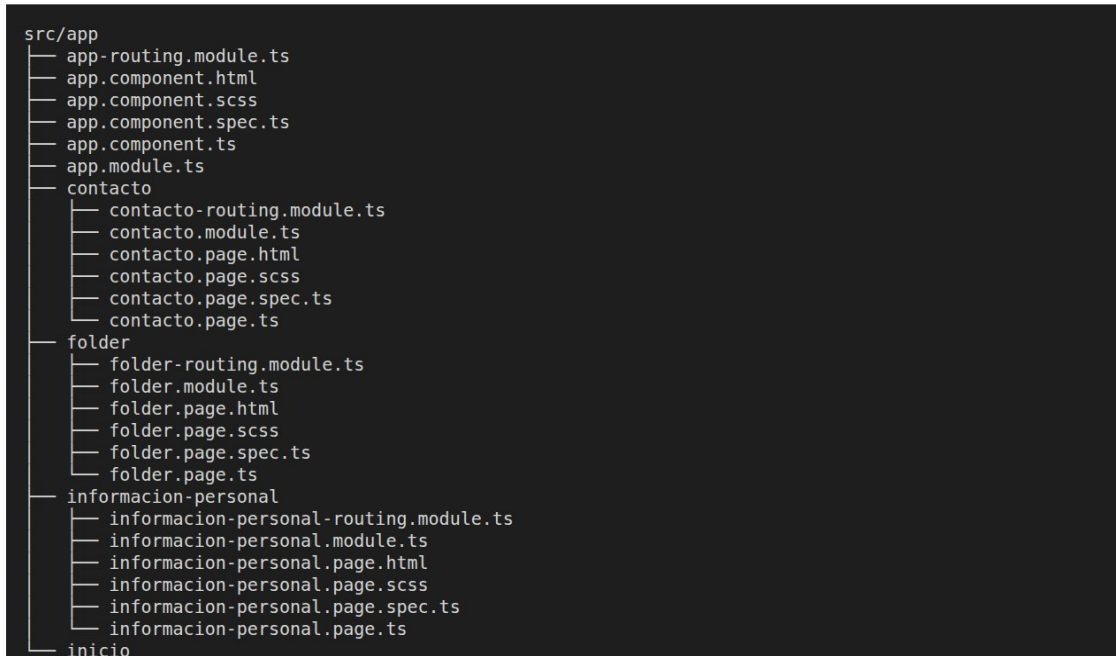


Foto II. Estructura de archivos tras la generación de las tres páginas independientes. Autor: Alcides Mata

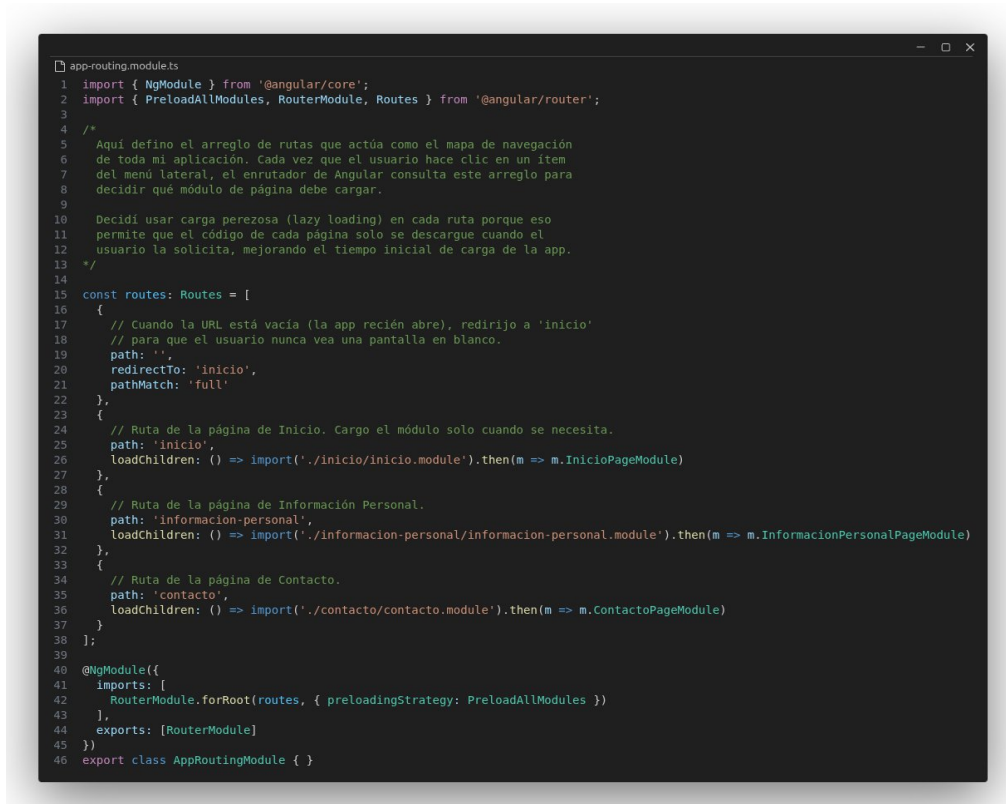
Una vez generadas las páginas, se procedió a eliminar la carpeta folder original que contenía la implementación basada en condicionales, dejando el proyecto con una arquitectura limpia donde cada sección del menú corresponde a su propia página independiente.

Fase 2: Estructura de Rutas y Navegación

1. Configuración del Módulo de Enrutamiento

Una de las decisiones técnicas más relevantes del proyecto fue la implementación de carga perezosa (lazy loading) para cada una de las tres rutas. Esta técnica consiste en que el código de cada página solo se descarga en la memoria del dispositivo cuando el usuario solicita visitarla, en lugar de cargarse todo de manera simultánea al iniciar la aplicación. Esto mejora significativamente el tiempo de carga inicial, especialmente en dispositivos móviles con conexiones limitadas.

La configuración del archivo `app-routing.module.ts` quedó estructurada de la siguiente manera: se definió una ruta vacía que redirige automáticamente a `/inicio` para evitar que el usuario vea una pantalla en blanco al abrir la app, y se añadieron tres rutas con carga perezosa apuntando a los módulos de cada página:



```
app-routing.module.ts
1 import { NgModule } from '@angular/core';
2 import { PreloadAllModules, RouterModule, Routes } from '@angular/router';
3
4 /*
5  Aquí defino el arreglo de rutas que actúa como el mapa de navegación
6  de toda mi aplicación. Cada vez que el usuario hace clic en un ítem
7  del menú lateral, el enrutador de Angular consulta este arreglo para
8  decidir qué módulo de página debe cargar.
9
10 Decidí usar carga perezosa (lazy loading) en cada ruta porque eso
11 permite que el código de cada página solo se descargue cuando el
12 usuario la solicita, mejorando el tiempo inicial de carga de la app.
13 */
14
15 const routes: Routes = [
16   {
17     // Cuando la URL está vacía (la app recién abre), redirijo a 'inicio'
18     // para que el usuario nunca vea una pantalla en blanco.
19     path: '',
20     redirectTo: 'inicio',
21     pathMatch: 'full'
22   },
23   {
24     // Ruta de la página de Inicio. Cargo el módulo solo cuando se necesita.
25     path: 'inicio',
26     loadChildren: () => import('../inicio/inicio.module').then(m => m.InicioPageModule)
27   },
28   {
29     // Ruta de la página de Información Personal.
30     path: 'informacion-personal',
31     loadChildren: () => import('../informacion-personal/informacion-personal.module').then(m => m.InformacionPersonalPageModule)
32   },
33   {
34     // Ruta de la página de Contacto.
35     path: 'contacto',
36     loadChildren: () => import('../contacto/contacto.module').then(m => m.ContactoPageModule)
37   }
38 ];
39
40 @NgModule({
41   imports: [
42     RouterModule.forRoot(routes, { preloadStrategy: PreloadAllModules })
43   ],
44   exports: [RouterModule]
45 })
46 export class AppRoutingModule { }
```

Foto III. `app-routing.module.ts` junto con explicación de su funcionamiento. Autor: Alcides Mata

Paralelamente, en el archivo `app.component.ts` se actualizó el arreglo `appPages` que define los ítems del menú lateral. Cada ítem es un objeto con tres propiedades: el título visible para el usuario, la URL a la que navega y el icono de `Ionicons` que se muestra al lado. Esta estructura permite que el HTML del menú itere automáticamente sobre el arreglo y renderice los botones, evitando código repetitivo.

2. Verificación de Rutas en el Navegador

Una vez configurado el sistema de rutas, se levantó el servidor de desarrollo mediante el comando `ionic serve` y se procedió a verificar manualmente que cada una de las tres rutas respondiera correctamente. A continuación, se presentan las capturas de las tres rutas funcionando con su contenido placeholder antes de la implementación del diseño final:

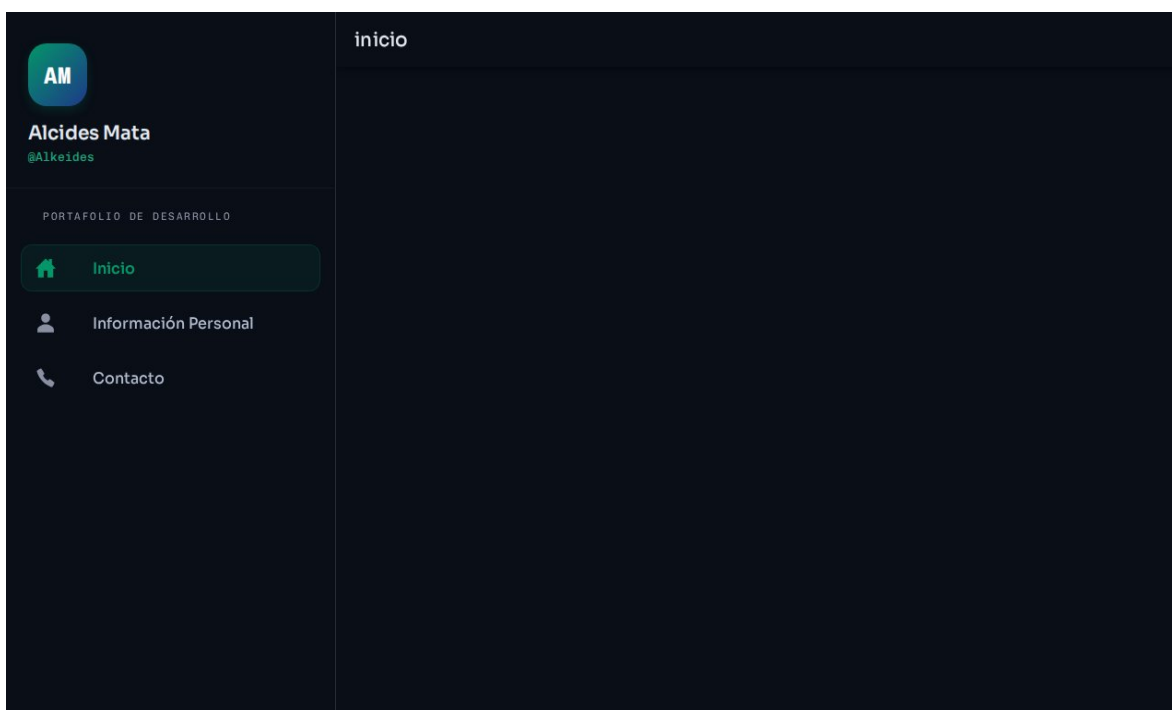


Foto IV. Verificación de la ruta /inicio funcional en el navegador. Autor: Alcides Mata

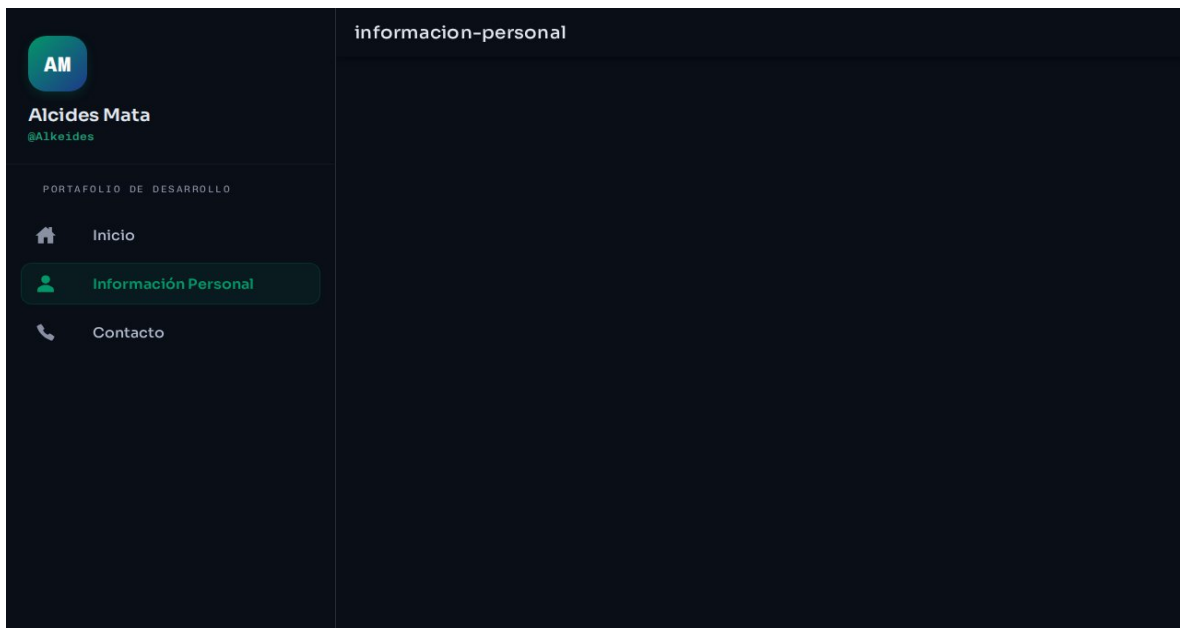


Foto V. Verificación de la ruta /informacion-personal funcional en el navegador. Autor: Alcides Mata

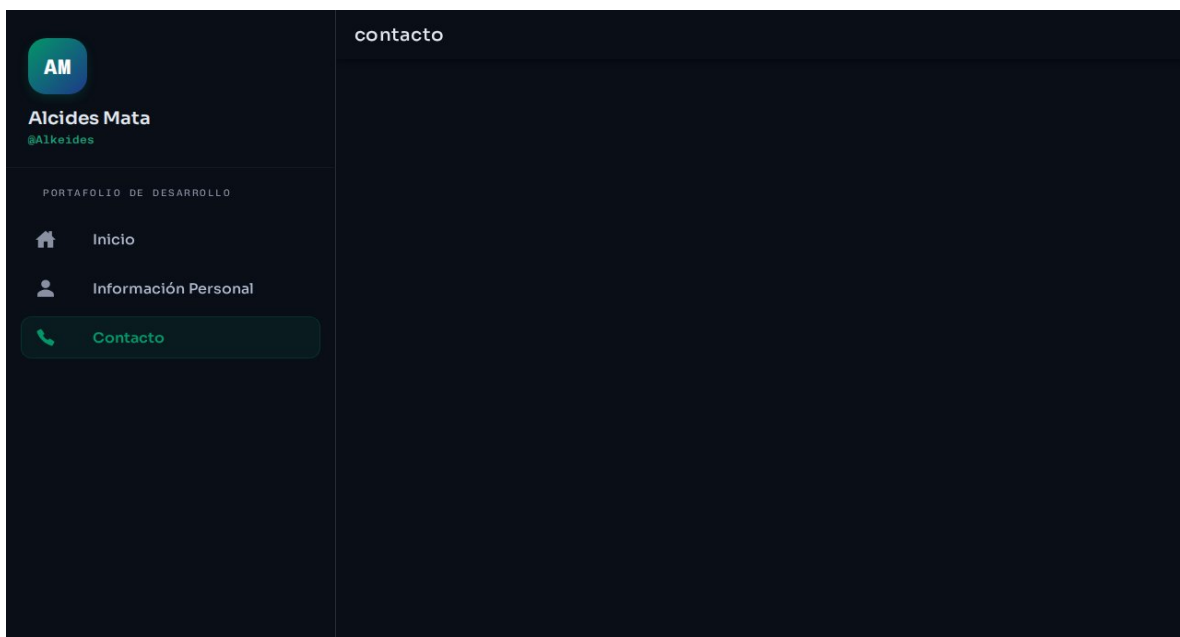


Foto VI. Verificación de la ruta /contacto funcional en el navegador. Autor: Alcides Mata

Con estas tres verificaciones se confirmó que el sistema de rutas estaba operando correctamente y que cada página podía ser accedida tanto a través del menú lateral como mediante la URL directa en el navegador. Esta validación previa fue importante para asegurar

que cualquier problema visual posterior estuviera relacionado con el diseño de cada página y no con la infraestructura de navegación.

Fase 3: Implementación de las Vistas

Esta fue la fase más extensa del proyecto, ya que involucró el diseño e implementación del contenido visual específico de cada una de las tres páginas. Para mantener una estética consistente y profesional, se siguió una dirección de diseño basada en minimalismo moderno con tipografía generosa, espaciado amplio y un acento de color sutil que aportara identidad sin recurrir a elementos decorativos innecesarios como gradientes complejos o efectos de cristal, alejandonos de la visión original del diseño de la web hace 2 semanas.

1. Página de Inicio

La página de Inicio fue diseñada como la primera vista que el usuario observa al abrir la aplicación, por lo que su propósito principal es presentar al usuario y ofrecer accesos rápidos a las otras secciones. Su estructura visual contiene los siguientes elementos:

- Un encabezado de Ionic con el título “Inicio” y el botón que despliega el menú lateral.
- Una sección central de bienvenida con el nombre del usuario destacado tipográficamente.
- Un subtítulo con el rol o profesión.
- Una frase descriptiva personal corta.
- Dos botones de navegación directa a las páginas de Información Personal y Contacto.

En el archivo inicio.page.ts se inyectó el servicio Router de Angular para programar la navegación desde los botones de la vista hacia las otras páginas. La lógica del componente es deliberadamente mínima, ya que el contenido es estático y no requiere consumir servicios externos.

A continuación, se presenta la captura de la página de Inicio con el diseño final aplicado:



Foto VII. Página de Inicio con diseño final aplicado. Autor: Alcides Mata

2. Página de Información Personal

La página de Información Personal fue concebida como una tarjeta de presentación digital donde el usuario puede mostrar su perfil profesional. Esta es la página con mayor densidad de contenido del proyecto, ya que organiza varios bloques de información en una sola pantalla manteniendo legibilidad y jerarquía visual. Su estructura contiene:

- Encabezado de Ionic con el título “Información Personal”.
- Avatar circular en la parte superior central que muestra la foto de perfil del usuario.
- Nombre completo y rol profesional debajo del avatar.
- Sección “Sobre mí” con biografía corta.
- Sección “Datos Académicos” con carrera, universidad y año en curso.
- Sección “Intereses” con una lista de habilidades o áreas de interés presentadas mediante el componente ion-chip.

Para la imagen de perfil se utilizó el componente ion-avatar de Ionic, que aplica automáticamente un recorte circular y manejo responsivo. La imagen se carga desde la

carpeta src/assets/, lo que permite reemplazar fácilmente la fotografía sin modificar el código del componente.

Una decisión de diseño importante fue evitar la anidación de tarjetas (cards dentro de cards), un anti-patrón común que genera ruido visual sin aportar jerarquía real. En su lugar, se utilizaron secciones separadas por espaciado vertical, lo que mantiene la información organizada sin saturar la pantalla.

A continuación, se presenta la captura de la página de Información Personal con el diseño final aplicado:

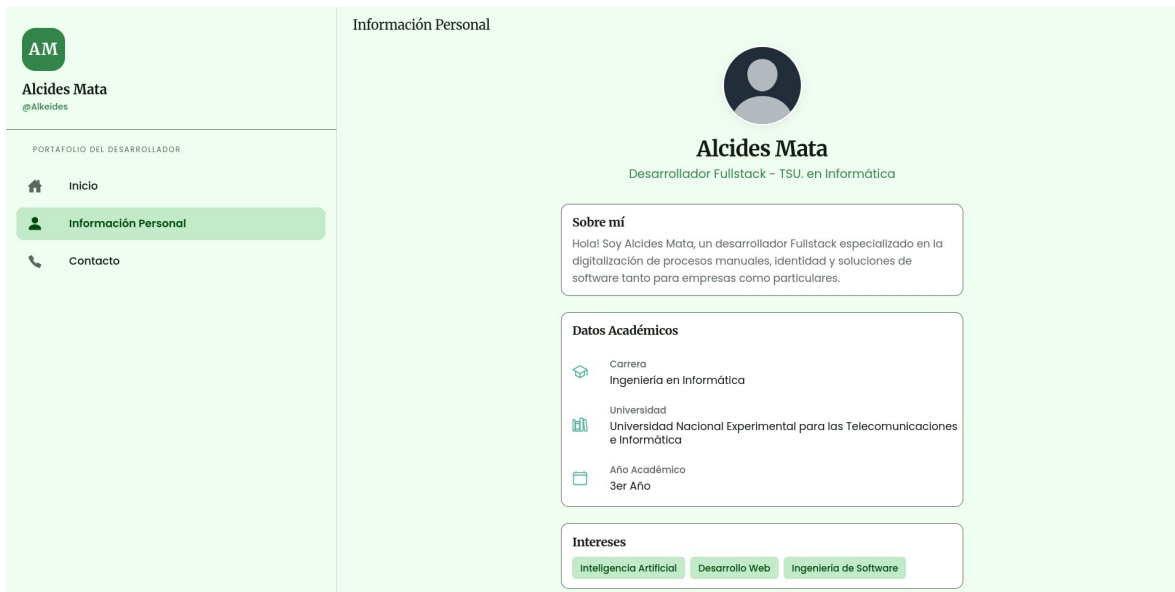


Foto VIII. Página de Información Personal con diseño final aplicado. Autor: Alcides Mata

3. Página de Contacto

La página de Contacto reúne los canales por los cuales el usuario puede ser contactado. El reto en su diseño fue evitar que se sintiera como un formulario corporativo frío, por lo que se optó por presentar la información como una lista clara y amigable. Su estructura contiene:

- Encabezado de Ionic con el título “Contacto”.
- Una breve introducción textual que invita al contacto.
- Una lista de canales de contacto utilizando los componentes ion-list e ion-item, cada uno con su icono correspondiente al lado izquierdo:
 - o Correo electrónico (icono mail-outline)
 - o Número telefónico (icono call-outline)
 - o Ubicación geográfica (icono location-outline)
 - o LinkedIn (icono logo-linkedin)
 - o GitHub (icono logo-github)
- Un formulario simple con campos de entrada para nombre, correo y mensaje, más un botón de “Enviar”. (Meramente estético).

A continuación, se presenta la captura de la página de Contacto con el diseño final aplicado:



Foto IX. Página de Contacto con diseño final aplicado. Autor: Alcides Mata

Los iconos utilizados pertenecen al conjunto Ionicons, integrado de forma nativa con Ionic Framework, lo cual garantiza consistencia visual sin necesidad de incluir librerías externas. El formulario fue construido como un elemento puramente de interfaz, sin lógica de envío backend, ya que el alcance de la actividad no contemplaba conexión a servicios externos.

Fase 4: Validación Responsiva en Dispositivos Móviles

Dado que Ionic está orientado al desarrollo de aplicaciones móviles, era imprescindible verificar que las tres páginas mantuvieran su legibilidad, proporciones y comportamiento adecuado al ser visualizadas en pantallas pequeñas. Para realizar esta validación se utilizó el modo responsivo de las herramientas de desarrollador del navegador, simulando un dispositivo iPhone estándar. A continuación, se presentan las tres capturas correspondientes a la vista móvil de cada página:



Foto X. Vista responsiva móvil de la Página de Inicio. Autor: Alcides Mata

 Información Personal



Alcides Mata
Desarrollador Fullstack - TSU. en
Informática

Sobre mí

Hola! Soy Alcides Mata, un desarrollador Fullstack especializado en la digitalización de procesos manuales, identidad y soluciones de software tanto para empresas como particulares.

Datos Académicos

 Carrera
Ingeniería en Informática

 Universidad
Universidad Nacional Experimental para las Telecomunicaciones e

Foto XI. Vista responsiva móvil de la Página de Información Personal. Autor: Alcides Mata



Foto XII. Vista responsiva móvil de la Página de Contacto. Autor: Alcides Mata

En las tres capturas se puede observar que el menú lateral, los componentes de Ionic y la tipografía mantienen su comportamiento adecuado en pantallas pequeñas. El espaciado se ajusta automáticamente, los iconos conservan su proporción y la información se mantiene legible sin requerir desplazamiento horizontal. Esto confirma que los componentes nativos de Ionic gestionan correctamente la adaptación responsiva sin necesidad de reglas CSS adicionales para los breakpoints comunes de dispositivos móviles.

Con esto, se da por concluido el desarrollo de la aplicación, habiendo cumplido con los tres requerimientos planteados en la asignación: la creación de los tres menús (Inicio, Información Personal y Contacto), la implementación de cada uno como una página independiente con su propio módulo, y la documentación del código fuente con comentarios propios en primera persona que explican las decisiones técnicas tomadas durante la implementación.

CONCLUSIÓN

La realización de esta evaluación práctica permitió consolidar conocimientos fundamentales sobre el desarrollo de aplicaciones móviles híbridas mediante el uso de Ionic Framework integrado con Angular. A través del proceso de refactorización del proyecto inicial, se comprobó que la generación de páginas independientes mediante Ionic CLI no solo cumple con un requisito académico, sino que aporta beneficios reales en términos de mantenibilidad, organización del código y rendimiento, especialmente cuando se combina con técnicas como el lazy loading.

La implementación de un menú lateral con tres vistas distintas demostró que Ionic ofrece un sistema de componentes maduro y consistente, donde elementos como ion-menu, ion-router-outlet, ion-list o ion-card funcionan de manera coordinada sin requerir configuración compleja. Asimismo, el uso de Ionicons eliminó la dependencia de librerías externas para los iconos, manteniendo el proyecto ligero y consistente visualmente.

El proceso de refactorización también permitió observar la diferencia práctica entre construir tres vistas dentro de una sola página mediante condicionales (la primera versión del proyecto) y construir tres páginas verdaderamente independientes con sus propios módulos (la versión final). Si bien ambas soluciones producen un resultado visualmente similar para el usuario final, la segunda demuestra dominio sobre la arquitectura de Ionic y prepara el proyecto para escalar de manera ordenada en futuras evaluaciones.

Finalmente, esta experiencia refuerza la importancia de documentar el código con comentarios propios. Al explicar el porqué de cada decisión, se transforma el código en un registro vivo del proceso de aprendizaje, no solo de su resultado. Esta práctica forma parte del stack de habilidades que considero fundamentales para mi desarrollo como ingeniero en formación, junto con el uso responsable de herramientas modernas que aceleren el proceso de implementación sin desplazar el entendimiento de lo que se construye.

ANEXOS

[Enlace al Github](#)

REFERENCIAS

Ionic Team. (s.f.). Ionic Framework: Cross-Platform Mobile App Development. Recuperado el 7 de mayo de 2026, de <https://ionicframework.com/docs>

Ionic Team. (s.f.). Ionic CLI. Recuperado el 7 de mayo de 2026, de <https://ionicframework.com/docs/cli>

Ionic Team. (s.f.). Ionicons: Premium designed icons for use in web, iOS, Android, and desktop apps. Recuperado el 7 de mayo de 2026, de <https://ionic.io/ionicons>

Google. (s.f.). Angular Documentation. Angular. Recuperado el 7 de mayo de 2026, de <https://angular.dev/overview>

Google. (s.f.). Angular Router: Lazy Loading Feature Modules. Angular. Recuperado el 7 de mayo de 2026, de <https://angular.dev/guide/ngmodules/lazy-loading>

Ionic Team. (s.f.). Capacitor: Cross-platform native runtime for web apps. Recuperado el 7 de mayo de 2026, de <https://capacitorjs.com/docs>

Universidad Nacional Experimental de las Telecomunicaciones e Informática. (s.f.). Programación III: Exploremos IONIC [Material del aula virtual]. Ministerio del Poder Popular para la Educación Universitaria. <https://www.uneti.edu.ve/campus/mod/book/view.php?id=54958>